Dipartimento di Elettronica e Informazione

Politecnico di Milano
prof. ***Carlo Ghezzi***
prof. ***Angelo Morzenti***
prof. ***Pierluigi San Pietro***

20133 Milano (Italia)
Piazza Leonardo da Vinci, 32
Tel. (39) 02-2399.3400
Fax (39) 02-2399.3411

Appello di Ingegneria del Software

8 Settembre 2004

Cognome

Nome

Matricola

Sezione (segnarne una) Ghezzi ☐, Morzenti ☐, SanPietro ☐

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h15m.
6. Punteggio totale a disposizione: 26/30.

Valutazione:
Esercizio 1: ……..

Esercizio 2: ……..

Esercizio 3: ……..

Esercizio 1 (punti 10)

La classe PilaDiInteriFiniti realizza una pila di capacità limitata i cui elementi sono numeri interi compresi tra 1 e 9 (estremi inclusi). Nell'implementazione della classe si decide di rappresentare la pila come un intero positivo in base 10: ad esempio, il numero 1325 rappresenta la pila in cui sono stati inseriti, nell'ordine, i numeri 5, 2, 3 e 1. Di conseguenza, l'operazione che esegue top produrrebbe 1, l'operazione pop produrrebbe 325, mentre push(7) produrrebbe 71325.

1. Si completi innanzitutto la **specifica** della classe, rispondendo anche alle domande che in essa appaiono. Si deleghi ai clienti del modulo il rispetto del vincolo che i valori da inserire nella pila debbano essere compresi tra 1 e 9. Il tentativo di inserimento in una pila al limite della capacità provoca il lancio dell'eccezione PilaPienaException.

```
public class PilaDiInteriFiniti {
// OVERVIEW: una PilaDiInteriFiniti tipica e'   mutabile o immutabile ? mutabile
//Una PilaDiInteriFiniti  tipica e' <In In-1 ...I1> dove In rappresenta l'intero inserito piu' recentemente;
// <> e' pertanto la pila vuota
// La pila contiene solo numeri tra 1 e 9
    public PilaDiInteriFiniti()
    //EFFECTS: costruisce una pila vuota

    public void push (int n)  throws PilaPienaException;
    /REQUIRES: n >0 && n<=9
    //EFFECTS: if this è piena throw PilaPienaException,
    //else, posto this = <n1 n2 … nk>, k>=0, this_post = <n n1 n2 … nk>
    //MODIFIES this

    public int top() throws EmptyException;
    //EFFECTS: if this è vuota throw EmptyException
    //else, posto this = <n1 n2 … nk>, k>=1, n1 è restituito

    public int pop() throws EmptyException;
    //EFFECTS: if this è vuota throw EmptyException
    //else, posto this = <n1 n2 … nk>, k>=1, n1 è restituito e this_post = <n2 … nk>
    //MODIFIES this

    public int length()
    //EFFECTS: restituisce il numero di elementi in this
}
```

2. Facendo riferimento all'implementazione che abbiamo sopra tratteggiato e alla PilaDiInteriFiniti tipica definita nella specifica, utlizzando le seguenti dichiarazioni in Java che definiscono la rappresentazione (rep) di PilaDiInteriFiniti:

   private long codifica: //contiene l'intero che rappresenta la pila

   private int numeroCifre; //contiene il numero corrente di cifre della codifica

NB: il valore massimo per un valore di tipo long è *9,223,372,036,854,775,807*

a) si **definisca** la funzione di astrazione.

```
// introduciamo una funziona ausiliaria: cifra(n,i) restituisce l'i-esima cifra decimale
    // del numero n, n>=0, i>=1.
    //AF(c) =if c.numeroCifre >0 return <cifra(c.codifica, c.numeroCifra)
    //cifra(c.codifica, c.numeroCifra -1), ..., cifra(c.codifica,1)> else return <>
```

b) si **definisca** e si **implementi** un Rep Invariant RI per la classe PilaDiInteriFiniti.

```
    //RI(c) = c.numeroCifre>=0 && c.codifica>=0 &&
    //  if c.numeroCifre>0 then (c,codifica ha esattamente c.numeroCifre cifre decimali
    //  && tutte le cifre di c.codifica sono comprese tra 1 e 9) else c.codifica == 0
    //  (si noti che abbiamo scelto di rappresentare la pila vuota ponendo sia c.numeroCifre
    //  che c.codifica a zero; altre soluzioni sono possibili, ad esempio ponendo solo
    //  c.numeroCifre a zero. Ovviamente l'implementazione dei metodi deve essere
    //  consistente con la rappresentazione scelta)

public boolean repOk() {
    int k, val;

    if (numeroCifre < 0 || codifica < 0) return false;
    if (numeroCifre ==0) return (codifica == 0);
    //da adesso in poi controlli per numeroCifre > 0
    //Verifichiamo che codifica abbia esattamente numeroCifre cifre decimali
    if (codifica < 10^(numeroCifre – 1) ||
       codifica > (10^numeroCifre) – 1) return false;
    //Verifichiamo che ogni cifra decimale sia compresa tra 1 e 9
    val = codifica;
    for (k = 1; k <= numeroCifre; k++) {
        if (val % 10 == 0) return false; //ovviamente occorre solo controllare che sia != 0
        val = val / 10;
    return true;
}
```

3. Fornire l'implementazione del costruttore e dei metodi push, pop e length e dimostrare che per essi RI è effettivamente invariante.

```
public class PilaDiInteriFiniti {
    private long codifica: //contiene l'intero che rappresenta la pila
    private int numeroCifre; //contiene il numero di cifre della codifica

    public PilaDiInteriFiniti () {
        numeroCifre=0; codifica = 0;
    }

    public void push (int n) throws PilaPienaException {
        if (codifica>= 99,999,999,999,999,999) throw new PilaPienaException();
        codifica = codifica + n* (10^numeroCifre); numeroCifre++;
    }

    public int pop()throws EmptyException {
        if (codifica ==0) throw new EmptyException();
        numeroCifre--;
        int ris = codifica / (10^numeroCifre)
        codifica =codifica % (10^numeroCifre);
        return ris;
    }

    public int length() {
        return numeroCifre;
    }
}
```

4. Si vuole definire nella classe PilaDiInteriFiniti un metodo appendStack tale per cui date, ad esempio, le due stack s1: <1 7 4> e s2: <2 1 5 6> il risultato è <1 7 4 2 1 5 6>. *Spiegare* quali sono le differenze fra i due modi seguenti di realizzazione del metodo, precisandone anche la dichiarazione:
    a) La funzione viene invocata scrivendo s1.appendStack(s2). Ad esempio:
        s1= s1.appendStack(s2);

    b) La funzione viene invocata scrivendo appendStack (s1, s2). Ad esempio:
        s1=appendStack (s1, s2);

//in class PilaDiInteriFiniti:
//caso a:

```
public PilaDiInteriFiniti appendStack(PilaDiInteriFiniti s) throws PilaPienaException{ … }
```

//caso b:

```
public static PilaDiInteriFiniti appendStack(PilaDiInteriFiniti s1, PilaDiInteriFiniti s2)
    throws PilaPienaException { … }
```

Il secondo caso differisce dal primo per il fatto che il metodo viene dichiarato static e che necessita di un parametro in più (il primo metodo opera implicitamente su this). Abbiamo ipotizzato che il metodo possa sollevare una eccezione di tipo PilaPienaException, che viene sollevata nel caso in cui la concatenazione delle due pile dia una pila di dimensioni più grandi rispetto alle dimensioni massime della pila.

5. Si definiscano casi di test per la funzione appendStack (nel caso (b)) utilizzando un metodo black box.

Dal momento che l'esercizio non fornisce una specifica completa per il metodo appendStack, sulla quale basare il nostro test black box, risolviamo l'ambiguità del problema ipotizzando che il metodo abbia REQUIRES: s1 != null and s2!= null, e che la sua specifica sia strutturata come precedentemente detto al punto 4. Un test black box, allora, potrebbe contenere i seguenti casi:

a) **s1 e s2 sono entrambi vuoti;**
b) **s1 è vuoto, s2 no;**
c) **s2 è vuoto, s1 no;**
d) **s1 e s2 entrambi non vuoti;**
e) **i casi a) e d), ma con s1 e s2 alias.**

La specifica del metodo appendStack dice che il metodo solleva l'eccezione PilaPienaException, ma non è specificato quale sia la dimensione massima della pila, quindi non ci è possibile definire casi di test che coprano il caso di pila risultatnte troppo grossa.

Esercizio 2 (punti 6)

Si consideri una classe X in cui appare il metodo m avente la seguente interfaccia

```
int m(float k) throws C;
```

Una sottoclasse SC ridefinisce il metodo nei modi qui di seguito elencati. Per ciascuno, si dica se il principio di sostituzione è rispettato, giustificando accuratamente la risposta (non basta scrivere SI/NO). Nel caso non lo fosse, mostrare un esempio in cui si dimostra la non sostituibilità, spiegando perchè ciò fa sì che la ridefinizione non sia accettabile. Si ipotizzi che la specifica sia la stessa per tutti i metodi laddove questi non lanciano eccezioni.

1. int m(float k);

La ridefinizione rispetta la regola delle segnature, che consente di eliminare una o più eccezioni. Tuttavia, una tale modifica di solito viola la regola dei metodi, modificando la postcondizione del metodo. Il principio di sostituzione potrebbe tanto essere violato quanto non esserlo, a seconda di quali sono le postcondizioni di X.m e di SC.m. Se la specifica di X.m prevedesse, ad esempio, di lanciare un'eccezione di tipo C ogni volta che l'argomento k è 1, allora *nessuna* specifica di SC.m potrebbe in alcun modo verificare il principio di sostituzione. Se, invece, la specifica di X.m è nondeterministica (ad esempio, se affermasse che il metodo m può sollevare l'eccezione oppure no, ma non specifica in quali casi la solleva), allora "eliminando" il sollevamento dell'eccezione nella postcondizione di SC.m viene rafforzata la postcondizione, e il principio di sostituzione non viene violato.

2. int m(float k) throws C, K;

NO, in quanto K non era prevista nel metodo originario: viola la regola delle segnature,

3. int m(float k) throws C1; //C1 sottoclasse di C

SI, in quanto l'eccezione di tipo C1 è anche di tipo C (rispetta la regola delle signature), quindi la m ridefinita lancerà ancora eccezioni di tipo C, sia pure più specifiche (restringendo il numero di eccezioni che posso sollevare, rinforzo la postcondizione di m e quindi rispetto la regola dei metodi).

4. int m(float k) throws C1; //C1 superclasse di C

NO; in quanto il chiamante potrebbe vedersi restituire un'eccezione di tipo diverso da quella prevista.
Esempio: il metodo:

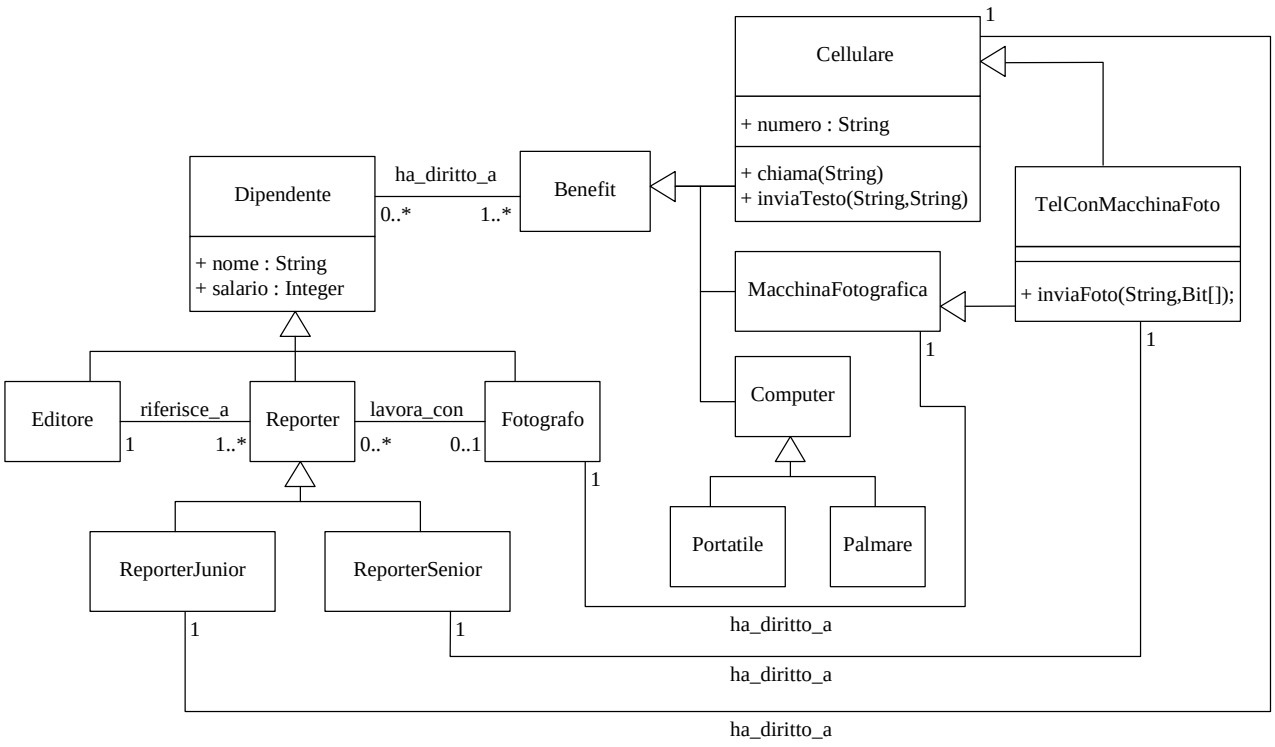
```
statici void prova(X x) {
try {
        x.m(1)
catch(C c)
```

se chiamato su un oggetto di tipo SC potrebb, eseguendo x.m(1) ritrovarsi un'eccezione di tipo C1, che non può essere catturata dal catch (C c).

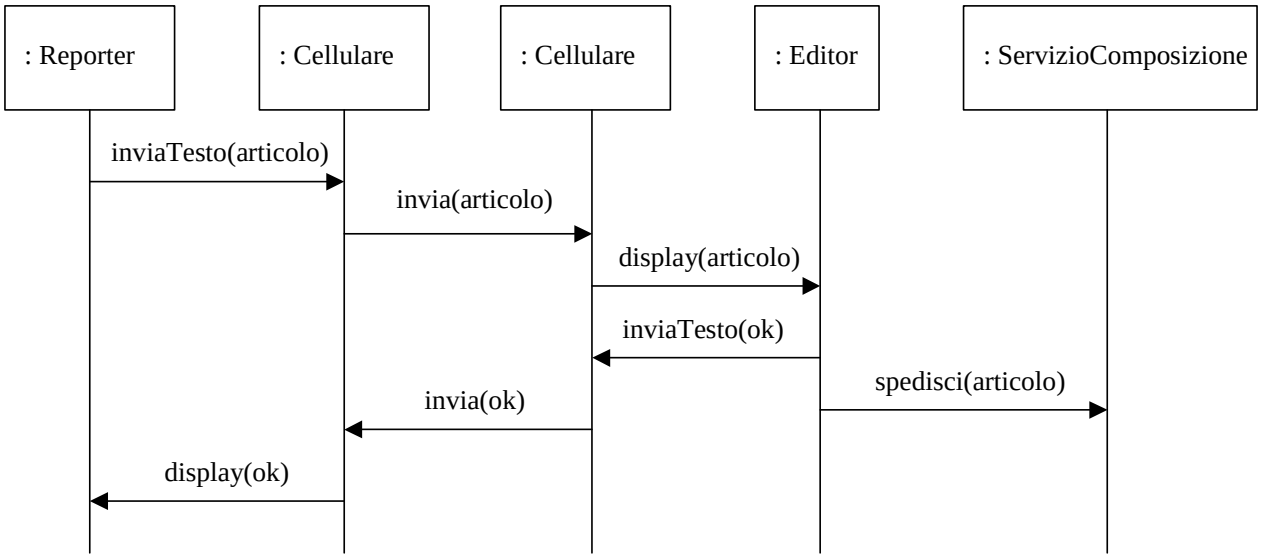
Esercizio 3 (punti 10)

Nella redazione di una testata giornalistica ci sono tre tipi di giornalisti: gli editori, i reporter, ed i fotografi. Ogni dipendente è caratterizzato da un nome e da un salario e ha diritto ad almeno un benefit (cioè un oggetto che viene concesso in uso al dipendente dall'azienda, ma che è di proprietà dell'azienda). Ci possono essere vari tipi di benefit: telefono cellulare, macchina fotografica, computer (che può essere o un portatile, o un palmare). Tra i benefit ci possono anche essere degli apparecchi che hanno funzionalità sia di telefono cellulare che di macchina fotografica.
Un telefono cellulare è caratterizzato da un numero di telefono, e offre la funzionalità di chiamata di un altro numero, e di spedizione di un testo ad un altro telefono. Se il telefono ha anche funzionalità di macchina fotografica, permette anche di inviare immagini (che si possono immaginare come sequenze di bit).
I fotografi hanno diritto, come benefit, ad esattamente una macchina fotografica.
Ci sono 2 tipi di reporter: i reporter junior e quelli senior. I reporter junior hanno diritto ad esattamente un telefono cellulare; i reporter senior hanno invece diritto, come benefit, ad un apparecchio con doppia funzionalità celullare/macchina fotografica.
Un reporter può lavorare in coppia con un fotografo, e fa riferimento ad un editor.

1. Scrivere un diagramma delle classi UML che rappresenti gli elementi della redazione descritti sopra.



2. Scrivere un sequence diagram UML che descriva il seguente svolgimento di eventi: un reporter spedisce, mediante telefono cellulare, un testo al suo editor, il quale lo controlla e manda al reporter la conferma dell'accettazione dell'articolo. L'editor, dopo aver confermato l'accettazione dell'articolo al reporter, manda l'articolo al servizio di composizione per l'inclusione nel giornale.



3. Si supponga di avere le seguenti interfacce CellulareI e MacchinaFotograficaI:

```
interface CellulareI {
      void chiama(String num);
      void inviaTesto(String num, String testo);
}

interface MacchinaFotograficaI {
      void scatta();
}
```

Si completino le seguenti dichiarazioni:

```
interface TelefonoConMacchinaFotograficaI extends CellulareI, MacchinaFotograficaI { }

class TelefonoCellulare implements CellulareI {
      public final numero String;
      void chiama(String num) {/* codice del metodo non mostrato */};
      void inviaTesto(String num, String testo) {/* codice del metodo non mostrato */};
}

class MacchinaFotografica implements MacchinaFotograficaI {
      void scatta() {/* codice del metodo non mostrato */};
}

class TelefonoConMacchinaFotografica extends TelefonoCellulare, implements MacchinaFotograficaI {
      void scatta() {/* codice del metodo non mostrato */};
}
```